

FIT HUB CRM – FITNESS CENTER MANAGEMENT



Salesforce CRM Implementation Documentation Guidelines

Professional standards for complete, auditable
Salesforce project documentation

ORGANIZATION / AUTHOR: [Your Name or Team]

DATE: [Insert Date]

Why Project Documentation Matters

-  **Single source of truth:** purpose, design, development, and deployment captured end-to-end
-  **Aligns business requirements:** directly connects defined user needs to implemented functionality
-  **Blueprint for admins & developers:** acts as a guide ensuring consistent development and scalability
-  **Supports training & troubleshooting:** provides detailed insights into every component and its logic
-  **Enables robust governance:** facilitates change management, audit compliance, and knowledge transfer
-  **Drives long-term sustainability:** acts as an essential asset for the controlled evolution of the CRM

General Submission Instructions

- ✓ **Format:** Submit the documentation in a professional format (preferably in Word or PDF).
- ✓ **Structure:** Use clear headings, subheadings, and bullet points for easy reading.
- ✓ **Typography:** Follow a consistent font style and size (e.g., Times New Roman, size 12 or 13).
- ✓ **Quality:** Ensure no grammatical mistakes and proper alignment of all sections.
- ✓ **Originality:** Plagiarism is strictly prohibited — your content must be 100% original.
- ✓ **Scope of Detail:** Provide only necessary details to create components. Avoid step-by-step implementation instructions.
- ! **Mandatory Evidence:** Include screen captures of Salesforce Setup/configurations where relevant for *each and every* feature.

Project Overview

The Horizon Hotel Booking & Customer Management CRM is a centralized Salesforce application designed to manage guest relationships, streamline room reservations, and automate front-desk operations. This project transitions the hotel from disjointed legacy systems to a unified cloud platform. Key features include end-to-end reservation tracking, automated guest communications, dynamic room availability monitoring, and customized reporting. The primary business need is to eliminate manual booking errors, reduce check-in times, and provide a 360-degree view of guest preferences to enhance customer service.

Objectives

The main goal of building this CRM is to create a seamless, efficient, and highly automated reservation lifecycle while empowering staff with actionable guest insights. By implementing this system, the business will achieve faster booking resolutions, improved customer retention through personalized service, and precise revenue tracking. These objectives directly link to business value by reducing operational overhead, streamlining bookings, and ultimately increasing the hotel's bottom line through optimized room occupancy.

Mandatory Sections: Overview and Objectives

Project Overview

Provide a brief paragraph summarizing what your CRM project is about.

- Highlight the **key features** and core functionality.
- Define the specific **business needs** being addressed (e.g. lead-to-cash, simplified bookings, enhanced support).
- Set the context for the system's purpose and primary users.

Objectives

Write a clear paragraph stating the main goals of building the CRM.

- Link objectives to **business value** (e.g., better customer management, streamlined workflows).
- Identify **measurable outcomes** and KPIs (e.g., cycle time reduction, adoption rates).
- Ensure alignment with overall strategic organizational initiatives.

Phase 1: Requirement Analysis & Planning

Understanding Business Requirements

The hotel staff previously relied on spreadsheets and manual emails, leading to double-booked rooms and lost guest data. The core requirements include solving these pain points by establishing a centralized guest database, automating confirmation emails, and providing real-time visibility into room status across all departments.

Defining Project Scope and Objectives

- * Scope: Implementation of Sales Cloud concepts, custom objects for Room and Reservation management, and automation for standard operational tasks.
- * Out of Scope: Integration with third-party payment gateways (planned for Phase 2).
- * Objectives:
 - * Achieve 100% digital tracking of bookings.
 - * Reduce manual data entry by 40% using automation.
 - * Establish secure data access protocols based on user roles.

Design Data Model and Security Model

- * Data Model: Utilizes standard objects (Account, Contact, Opportunity renamed to Booking) and custom objects (Room_c, Guest_Review_c). Lookups and Master-Detail relationships link Bookings to specific Rooms and Contacts.

* Security Model: Organization-Wide Defaults (OWD) set to Private for Bookings and Guest Reviews to ensure data privacy. Standard Profiles are customized to restrict access based on staff roles.

Stakeholders Mapping

- * Executive Sponsor: General Manager (Focus: ROI, Revenue Dashboards).
- * Process Owners: Front Desk Manager, Sales Director (Focus: Daily operations, Lead conversions).
- * End Users: Front Desk Agents, Customer Service Reps (Focus: Ease of use, speed).

Execution RoadMap

- * Week 1: Requirements gathering, Data modelling, and Setup.
- * Week 2: Backend configuration, Apex development, and Automation.
- * Week 3: UI/UX customization, LWC development, and Reporting.
- * Week 4: Data migration, UAT Testing, and Final Deployment.



Phase 2: Salesforce Development - Backend & Configurations

Setup Environment & Develops Workflow

Development was conducted in a dedicated Developer Pro Sandbox. Version control was managed using Git, tracking metadata changes. Deployments to the UAT environment were handled via Salesforce CLI and standard Change Sets.

Customization of Objects, Fields, Validation Rules, Automation

- * Objects & Fields: Created custom fields on the Booking object such as Check_In_Date_c, Check_Out_Datec, and Total_Amount_c (Roll-up Summary).

- * Validation Rules: * Check_Out_Must_Be_After_Check_In: Ensures the check-out date is strictly greater than the check-in date.

- * Automation (Flows): * Record-Triggered Flow: Automatically sends a customized Welcome Email alert to the Contact when a Booking status changes to "Confirmed".

- * Approval Process: A multi-step approval process created for "VIP Discounts." Any booking applying a discount greater than 15% automatically routes to the Sales Manager for approval before confirmation.

> [Insert Screenshot: Custom Object Manager showing Room__c and Booking objects]

> [Insert Screenshot: Validation Rule logic and configuration]

> [Insert Screenshot: Record-Triggered Flow Builder canvas]

> [Insert Screenshot: Discount Approval Process steps]

>

Apex Classes, Triggers, Asynchronous Apex Classes

- * Apex Trigger (Room Status Trigger): A before update trigger on the Booking object. When a booking is marked as "Checked Out," the trigger automatically queries the related Room__c record and updates its status to "Needs Cleaning," ensuring real-time operational accuracy without manual input.

- * Asynchronous Apex (Batch): Developed a nightly Batch Apex class to identify "Pending" bookings older than 48 hours and automatically update their status to "Cancelled" to free up room availability.

> [Insert Screenshot: Developer Console showing the Room Status Trigger code]

> [Insert Screenshot: Batch Apex execution from Apex Jobs]

>

Phase 2: Development — Backend & Configurations

1) Configuration & DevOps

- **Environments:** Sandboxes strategy; version control and CI/CD setup documented.
- **Customization:** Objects, fields, page layouts, and validation rules logic defined.
- **Automation:** Flows (preferred approach), Workflow Rules/Process Builder (if legacy), Approval Processes.

</> 2) Apex Development (If Applicable)

- **Logic:** Apex classes, triggers, Invocable, Queueable, Batchable, and Schedulable Apex details.
- **Best Practices:** Bulkification approaches and governor limit considerations.
- **Quality:** Error handling mechanisms and code coverage targets.



Mandatory Requirement: Screenshots Evidence

Screenshots for **each Salesforce concept developed** are strictly mandatory. Your documentation must include visual evidence from Setup/Configuration for:

- Every created Object and Field Set
- Each Validation Rule
- Every Automation Flow or Approval Process
- All Apex Test executions.

Phase 3: UI/UX Development & Customization

Lightning App Setup

Created a custom Lightning App named "Horizon Hotel Management" via the App Manager, featuring a branded logo, customized utility bar (including Recent Items and Notes), and tailored navigation tabs highlighting Contacts, Bookings, and Rooms.

Page Layouts & Dynamic Forms

- * Upgraded the standard Booking page layout to use Dynamic Forms.
- * Configured conditional visibility: The "VIP Amenities Requested" section only appears on the UI if the Guest tier c field equals "Platinum" or "Gold."

User Management

Created distinct custom profiles: Front Desk Agent (Create/Read/Edit access to Bookings, Read-only for Reports) and Hotel Management (Modify All data, Access to Financial Dashboards).

Reports and Dashboards

- * Reports: "Current Month Revenue," "Upcoming Check-ins (Next 7 Days)," and "Average Room Occupancy."
- * Dashboards: Built an Executive Dashboard featuring gauge charts for monthly revenue targets and donut charts displaying room bookings by category (e.g., Deluxe, Suite).

LWC Development (Bonus)

Developed a custom Lightning Web Component (LWC) named Room availability calendar. This component retrieves real-time room status via an Apex controller and displays a visual, interactive 7-day calendar directly on the App Home Page, allowing staff to see available rooms at a glance.

Lightning Pages

Customized the Home Page with the newly developed LWC, a list view of "Today's Check-ins," and a Report Chart component showing recent sales.

- > [Insert Screenshot: Horizon Hotel App Navigation and Utility Bar]
- > [Insert Screenshot: Dynamic Form conditional visibility setup]
- > [Insert Screenshot: Front Desk Agent Profile settings]
- > [Insert Screenshot: Executive Dashboard UI]
- > [Insert Screenshot: Custom LWC Room Availability Calendar on Home Page]
- >



Phase 4: Data Migration, Testing & Security Data Migration Process

Legacy guest data (Accounts and Contacts) was imported using the Data Import Wizard to trigger standard duplicate rules during import. Bulk historical booking data

(over 50,000 records) was migrated using the Data Loader to ensure mapping accuracy for complex relationships.

Data Quality Tracking

- * Field History Tracking: Enabled on the Booking object specifically for the Status__c and Discount_Amount__c fields to maintain an audit trail.

- * Duplicate & Matching Rules: Configured custom Matching Rules on the Contact object (matching by First Name, Last Name, and Email) and activated Duplicate Rules to block the creation of duplicate guest profiles.

Security Configurations

- * Role Hierarchy: General Manager > Sales Manager > Front Desk Agent.

- * Permission Sets: Created a "Refund Processing" permission set assigned only to specific senior agents, granting them field-level access to the Refund_Amount__c field.

- * Sharing Rules: Created a Criteria-Based Sharing Rule to share Bookings marked as "Corporate" with the B2B Sales Public Group.

> [Insert Screenshot: Data Loader Success/Error files overview]

> [Insert Screenshot: Field History Tracking setup UI]

> [Insert Screenshot: Duplicate Rule configuration]

> [Insert Screenshot: Role Hierarchy tree]

>

Testing Approach and Test Cases

Testing was conducted using a mix of automated unit tests and manual UAT scenarios.

- * Test Classes: Created Room Status Trigger Test and Batch Cancel Pending Test using @isTest utility classes, achieving 92% overall code coverage using bulkified test data.

Test Case Documentation:

- * Test Case 1: Booking Creation & Validation

- * Input: Agent creates a Booking with Check-Out Date set prior to Check-In Date.

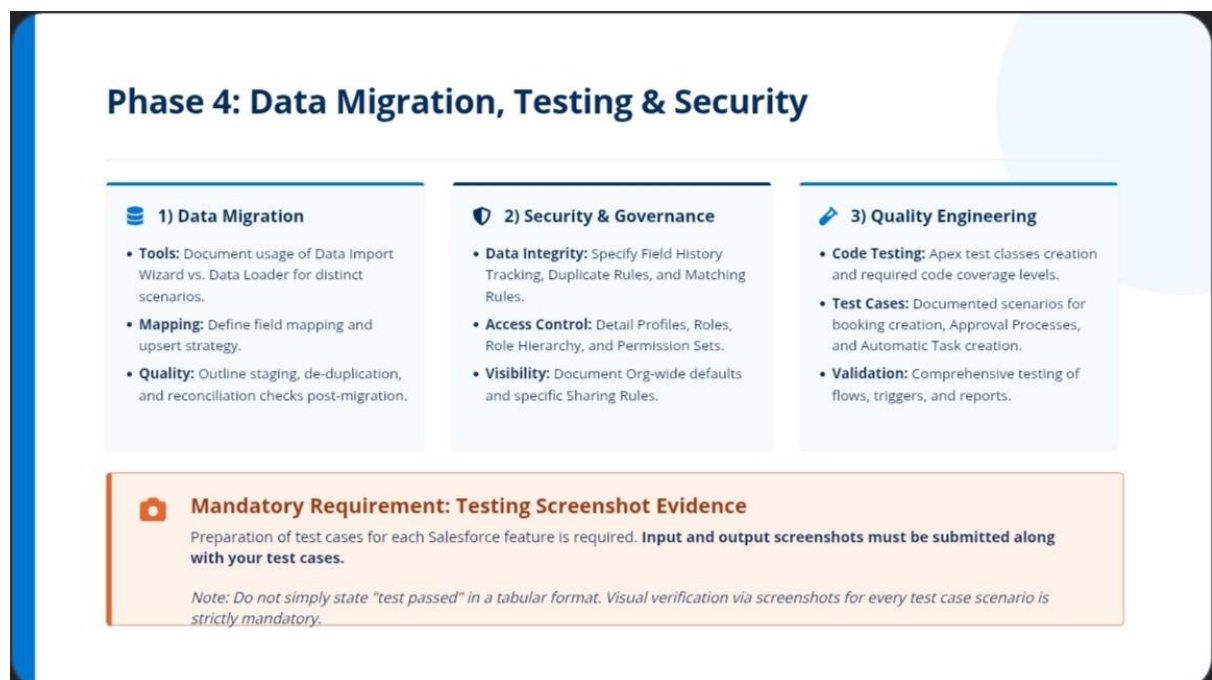
- * Output: System blocks save and displays the validation error message.

- * [Insert Screenshot: UI showing the Validation Error triggered]

- * Test Case 2: Approval Process Routing

- * Input: Agent applies a 20% discount on a \$1,000 suite booking.
- * Output: Record is locked; an approval request appears in the Manager's queue.
- * [Insert Screenshot: Record locked UI and Pending Approval history]
- * Test Case 3: Automated Welcome Email Flow
- * Input: Booking status is updated from "Draft" to "Confirmed."
- * Output: Flow triggers successfully; email log shows welcome message sent to Contact.
- * [Insert Screenshot: Email generated in Contact Activity Timeline]

Phase 5: Deployment, Documentation & Maintenance



Phase 4: Data Migration, Testing & Security

1) Data Migration	2) Security & Governance	3) Quality Engineering
Tools: Document usage of Data Import Wizard vs. Data Loader for distinct scenarios. Mapping: Define field mapping and upsert strategy. Quality: Outline staging, de-duplication, and reconciliation checks post-migration.	Data Integrity: Specify Field History Tracking, Duplicate Rules, and Matching Rules. Access Control: Detail Profiles, Roles, Role Hierarchy, and Permission Sets. Visibility: Document Org-wide defaults and specific Sharing Rules.	Code Testing: Apex test classes creation and required code coverage levels. Test Cases: Documented scenarios for booking creation, Approval Processes, and Automatic Task creation. Validation: Comprehensive testing of flows, triggers, and reports.

Mandatory Requirement: Testing Screenshot Evidence

Preparation of test cases for each Salesforce feature is required. **Input and output screenshots must be submitted along with your test cases.**

Note: Do not simply state "test passed" in a tabular format. Visual verification via screenshots for every test case scenario is strictly mandatory.

Phase 5: Deployment & Maintenance

Deployment Strategy

Deployment from the Developer Sandbox to the UAT Sandbox, and finally to Production, was executed using Change Sets. Dependencies were carefully mapped, deploying backend elements (Objects, Fields, Apex) in Change Set 1, followed by UI/UX elements (Layouts, LWC, Dashboards) in Change Set 2 to avoid missing reference errors.

System Maintenance and Monitoring

Post-deployment, the system is maintained via a weekly administrative review. The Salesforce Optimizer report is scheduled to run monthly to identify unused fields or redundant roles. Apex exception emails and Debug Logs are monitored continuously by the system admin to catch and resolve hidden backend errors.

Troubleshooting Approach

If users report issues (e.g., Flows not firing), the troubleshooting protocol involves:

- * Replicating the user's action via the "Login As" feature.
- * Reviewing specific record criteria against the Flow/Process conditions.
- * Utilizing the Developer Console to read Debug Logs for DML or governor limit exceptions.
- * Documenting the fix in the internal IT Wiki before pushing updates from Sandbox to Production.



Future Enhancements

To continuously improve the CRM, Phase 2 roadmaps include integrating an AI-powered Chatbot (Einstein Bots) on the hotel's website to automatically capture leads and create Contact records in Salesforce. Additionally, Einstein Activity Capture will be implemented to automatically log agent emails, and AI suggestions will be explored to recommend room upgrades based on guest history.

Conclusion

The Horizon Hotel Booking & Customer Management CRM implementation was a complete success, successfully transitioning the business to a modern, scalable cloud infrastructure. By aligning Salesforce's robust automation, rigorous security protocols, and custom UI components with the hotel's specific operational needs, the project has significantly reduced manual administrative work. The system now serves as a reliable single source of truth, ensuring the hotel is well-equipped to scale its operations, maintain data integrity, and deliver exceptional guest experiences for years to come.

Additional Points & Conclusion



1) Additional Documentation Points

- **Essential Details:** Provide necessary build details for objects, flows, etc., avoiding step-by-step implementation instructions.
- **Evidence Required:** Include setup/configuration screenshots for every Salesforce feature implemented.
- **Logic Description:** Briefly describe validation rules, approval processes, and automation flows created.
- **Testing Methodology:** Document the overall testing approach for flows, reports, Apex, and UI components.
- **Future Scope:** Note planned future enhancements such as chatbot integration, AI suggestions, or predictive insights.



2) Conclusion

Comprehensive documentation is an essential asset for long-term project viability.

By meticulously recording requirements, architecture, configuration, and testing approaches, teams can accelerate future delivery phases, ensure strict audit compliance, and enable sustainable, scalable growth of the Salesforce CRM platform.